

Detection Engineering Pipeline

Overview

This document outlines the design, implementation, and operational workflow of the Detection Engineering Pipeline used within a Splunk-based Security Operations Center (SOC) home lab environment.

The primary goal of this pipeline is to transform raw telemetry into actionable security detections while maintaining a centralized detection catalog that supports reporting, MITRE ATT&CK coverage analysis, adversary simulation validation, and dashboard visualization.

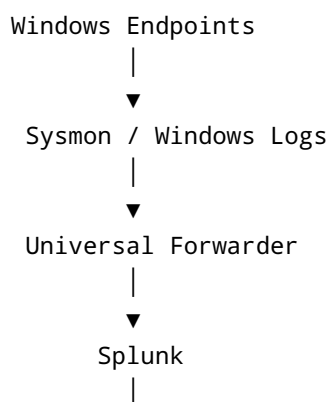
The architecture is designed to emulate enterprise SOC practices where detections are managed as structured security content rather than standalone search queries.

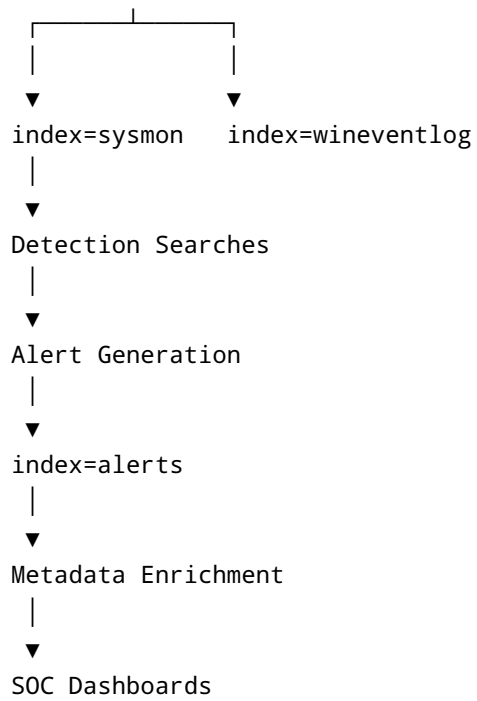
Objectives

The Detection Engineering Pipeline was developed to achieve the following objectives:

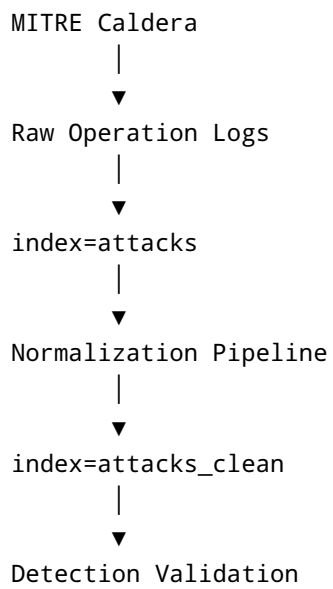
- Collect endpoint telemetry from Windows systems
- Execute detection logic against ingested data
- Generate structured security alerts
- Maintain a centralized detection inventory
- Map detections to MITRE ATT&CK techniques
- Measure detection coverage and effectiveness
- Validate detections through adversary emulation
- Support SOC dashboards and operational reporting

Architecture





Parallel adversary simulation workflow:



Splunk Index Design

index=sysmon

Stores endpoint telemetry generated by Sysmon.

Examples:

- Process Creation Events
- Network Connections
- Registry Modifications
- Driver Loads
- File Creation Events

Purpose

Provides detailed endpoint telemetry used by detection searches and threat hunting activities.

index=wineventlog

Stores Windows Security Event Logs.

Examples:

- Event ID 4624 – Successful Logon
- Event ID 4625 – Failed Logon
- Event ID 4720 – User Account Created
- Event ID 4732 – Group Membership Changes

Purpose

Supports authentication monitoring, privilege escalation detection, and Active Directory security use cases.

index=attacks

Stores raw adversary emulation activity generated by MITRE Caldera.

Purpose

Provides a source of truth for simulated attack activity used during detection validation.

index=attacks_clean

Stores normalized attack events derived from raw Caldera operation logs.

Purpose

Supports ATT&CK reporting, attack validation, and detection coverage analysis.

index=alerts

Stores detection results generated by Splunk searches.

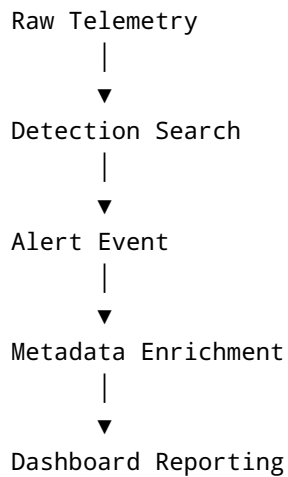
Purpose

Contains actionable security findings generated by detection content.

Raw telemetry should never be written directly to this index.

Detection Lifecycle

The detection lifecycle follows the workflow below:



Each stage adds context and value to the original telemetry, ultimately producing actionable security intelligence.

Detection Catalog

A centralized detection catalog is used to manage metadata associated with every detection.

The catalog is maintained through a CSV lookup table within Splunk.

Benefits

- Consistent ATT&CK mappings
- Standardized severity classifications
- Simplified dashboard development
- Centralized detection management
- Improved reporting and analytics

Detection Metadata Schema

Each detection contains the following metadata fields:

Field	Description
rule_id	Unique detection identifier
detection_name	Detection title
mitre_technique	Associated ATT&CK technique
severity	Alert severity level
status	Detection status
author	Detection author
created_at	Creation date
data_source	Telemetry source
log_source	Operating system source
false_positive	Expected false-positive level

Example

```
{
  "rule_id": "det_0021",
  "detection_name": "Suspicious PowerShell Execution",
  "mitre_technique": "T1059.001",
  "severity": "high",
```

```
"status": "active",  
"author": "Taji Abdullah",  
"created_at": "2026-06-08",  
"data_source": "Sysmon",  
"log_source": "Windows",  
"false_positive": "low"  
}
```

Creating the Detection Catalog

Create a CSV file named:

detections.csv

Upload the file through:

Settings
→ Lookups
→ Lookup Table Files

Create a lookup definition through:

Settings
→ Lookups
→ Lookup Definitions

Assign the lookup definition the following name:

detections_lookup

Testing the Detection Catalog

Verify that the lookup is accessible:

| inputlookup detections.csv

Expected fields:

```
rule_id
detection_name
severity
mitre_technique
```

Successful output confirms that the lookup has been configured correctly.

Alert Generation

Detection searches execute against telemetry indexes to identify suspicious activity.

Example Detection

```
index=sysmon EventCode=1 CommandLine="*-enc"
```

When a match is identified:

```
index=sysmon EventCode=1 CommandLine="*-enc"
| eval rule_id="det_0021"
| collect index=alerts
```

This creates a structured alert event within the `alerts` index.

Alert Enrichment

Alert events contain a detection identifier (`rule_id`) that is used to enrich alerts with metadata from the detection catalog.

```
index=alerts
| lookup detections_lookup rule_id OUTPUT *
```

Additional metadata fields become available:

- detection_name
- severity
- mitre_technique

- author
- data_source
- false_positive

This enrichment process provides analysts with additional context during investigations.

Scheduled Detection Execution

Detection searches should be configured as scheduled searches.

Recommended Schedule

Every 5 Minutes

Cron Expression

`* / 5 * * * *`

Recommended Search Window

`earliest=-5m latest=now`

Using a rolling five-minute window prevents unnecessary reprocessing of historical events.

Caldera Attack Data Pipeline

MITRE Caldera operation logs are written to:

`/tmp/event_logs/`

Events are stored as JSON arrays.

Example

```
[  
  { ... },  
  { ... },  
]
```



```
{...}  
]
```

These logs serve as the source data for attack validation and ATT&CK coverage analysis.

Attack Data Normalization

Raw attack logs must be normalized before they can be used for reporting and validation.

Normalization Search

```
source="/tmp/event_logs/*" host="SOC" index="attacks"  
| spath output=items path={}  
| mvexpand items  
| spath input=items  
| eval technique=attack_metadata.technique_id  
| eval technique_name=attack_metadata.technique_name  
| eval tactic=attack_metadata.tactic  
| eval host=agent_metadata.host  
| eval user=agent_metadata.username  
| eval command=plaintext_command  
| eval status=if(status==0,"success","failed")  
| eval _time=strptime(agent_reported_time,"%Y-%m-%dT%H:%M:%SZ")
```

This process extracts and standardizes relevant ATT&CK and host metadata.

Automated Attack Processing

A scheduled search executes every five minutes to process newly ingested attack data.

Normalized events are written to:

```
index=attacks_clean
```

This creates a continuously updated dataset for reporting and validation activities.

Detection Validation

Detection validation measures how effectively detections identify simulated adversary activity.

Attack Activity

```
index=attacks_clean
| stats count by technique
```

Detection Activity

```
index=alerts
| stats count by mitre_technique
```

Coverage Comparison

```
index=attacks_clean
| stats count as attacks by technique
| join technique [
  search index=alerts
  | stats count as detections by mitre_technique
  | rename mitre_technique as technique
]
```

This comparison highlights ATT&CK techniques that are executed but not detected, helping identify detection gaps.

Dashboard Integration

The Detection Engineering Pipeline supports multiple SOC dashboards.

SOC Overview Dashboard

Provides visibility into:

- Total event volume
- Alert volume
- Alert severity distribution
- ATT&CK activity
- Endpoint reporting status

Threat Detection Dashboard

Provides visibility into:

- Detection frequency
- Alert trends over time
- Severity distribution
- Top triggered detections

Adversary Simulation Dashboard

Provides visibility into:

- ATT&CK techniques executed
- Detection coverage metrics
- Detection gaps
- Attack timelines

Troubleshooting

Lookup Returns No Data

Verify that the lookup file is accessible:

```
| inputlookup detections.csv
```

If no results are returned, confirm that the lookup file and definition were created successfully.

Alerts Missing Metadata

Verify lookup enrichment:

```
| lookup detections_lookup rule_id OUTPUT *
```

Confirm that:

```
rule_id
```

exists in both the alert events and the lookup table.

Attack Data Not Appearing

Verify attack data ingestion:

```
source="/tmp/event_logs/*" index=attacks
```

Confirm that events are being indexed successfully.

Scheduled Searches Not Running

Review scheduled search configuration:

```
Settings  
→ Searches, Reports, and Alerts
```

Verify:

- Search is enabled
 - Cron schedule is correct
 - Time range is configured properly
-

Future Enhancements

Planned improvements include:

- Risk-Based Alerting (RBA)
- Detection scoring and prioritization
- SOAR integration
- Incident tracking workflows
- ATT&CK heatmaps
- Detection quality metrics
- Mean Time to Detect (MTTD) reporting
- Detection-as-Code automation
- Sigma-based CI/CD validation pipelines

Conclusion

This Detection Engineering Pipeline transforms raw telemetry into structured security findings while maintaining ATT&CK-aligned detection metadata and adversary validation capabilities.

The architecture demonstrates key cybersecurity disciplines including detection engineering, SIEM administration, ATT&CK mapping, adversary emulation, data normalization, dashboard development, and SOC operations. The design reflects workflows commonly found in enterprise security environments and provides a scalable foundation for continued detection development and validation.